```
    "title" : "The Godfather: Part II"
    ...
  }
}]
```

The output indicates that the query was executed in a query context because the query determined not only whether it matched a document but also how well the document was matched.

Why is the score for the second result (2.261362) slightly lower than the score for the first result (2.879596)? Because the engine's relevancy algorithm found "Godfather" in a title with two words ("the", "godfather"), and that match ranks higher than a match in a title of four words ("the","godfather","part","II").

> **NOTE** Queries on full-text search fields run in a query context because they are expected to have a score associated with each matched document.

Although fetching the result with a relevancy score is fine for most use cases, sometimes we do not need to find out how well the document matched. Instead, we just want to know whether there's a match. This is where the filter context comes into play.

FILTER CONTEXT

Let's rewrite the query from listing 8.3, but this time wrap our `match` query in a `bool` query with a `filter` clause.

Listing 8.4 A `bool` query with no score

```
GET movies/_search
{
  "query": {
    "bool": {
      "filter": [{
          "match": {
            "title": "Godfather"
          }
        }
      ]
    }
  }
}
```

In this listing, the results do not have a score (`score` is set to 0.0) because our query gives Elasticsearch a clue that it must be run in a filter context. If we are not interested in scoring the document, we can ask Elasticsearch to run the query in a filter context by wrapping the query in a `filter` clause, as we did here.

The main benefit of running a query in this context is that because Elasticsearch doesn't need to calculate scores for the returned search results, it can save some computing cycles. These filter queries are more idempotent, so Elasticsearch caches them for better performance.